

LLM-ERP Secrets Rotation SOP (English)

Leaked secrets = system fall. Regular rotation is the minimum. Audience: Customer IT, our ops, security consultants.



Contents

- [1. Secrets Catalog & Rotation Cycles](#)
 - [2. JWT_SECRET Rotation \(90 days\)](#)
 - [3. PostgreSQL Password Rotation \(6 months\)](#)
 - [4. LLM_API_KEY Rotation \(yearly / emergency\)](#)
 - [5. TLS Certificate Rotation \(90 days\)](#)
 - [6. WireGuard Public Key Rotation \(yearly\)](#)
 - [7. Emergency Response: Secret Leak](#)
 - [8. Automation + Monitoring](#)
-

1. Secrets Catalog & Rotation Cycles

Secret	Blast Radius	Rotation	Risk	Procedure
JWT_SECRET	All user tokens invalidated	90 days	High	§2
POSTGRES_PASSWORD	DB connection	Half-year	Medium	§3
LLM_API_KEY	API bill explosion	Yearly / emergency	High	§4
TLS cert	HTTPS	90 days (auto)	Low	§5
WireGuard pubkey	MESH VPN	Yearly	Medium	§6
LINE Channel secret	LINE Bot receiving	On change	Medium	LINE Console
SMTP credential	Email sending	Half-year	Low	Edit .env

2. JWT_SECRET Rotation (90 days)

2.1 Why Rotate

JWT is signed by `JWT_SECRET` . Once leaked, attacker can forge any identity.

2.2 Normal Rotation (no leak signal)

```
# 1. Generate new secret
NEW_SECRET=$(openssl rand -hex 32)
echo "New JWT_SECRET: $NEW_SECRET"

# 2. Back up old .env
cp backend/.env backend/.env.backup-$(date +%F)

# 3. Replace
sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$NEW_SECRET|" backend/.env

# 4. Restart backend
docker compose restart backend

# 5. Notify all users to re-login
# (all old JWTs invalidated)
```

2.3 Zero-Downtime Rotation (advanced: dual-key)

Implement JWT `kid` (key id) header to support multiple keys:

```
# Add kid="v2" to JWT payload
# Keep v1 key to decode old tokens, v2 to sign new ones
# After all v1 expire (24h), retire v1
```

Full implementation: Phase 2 roadmap. For now, use §2.2 simple rotation.

2.4 Notify Everyone

After JWT_SECRET rotation, **all old tokens immediately invalid:**

- Desktop UI: auto-redirects to login
- Mobile App: auto-logout (401 handler)
- LINE Bot: users rebind

Pre-broadcast:

```
[LINE Group Notice]
Tomorrow 02:00 we'll do security maintenance (JWT rotation).
All users just re-login. Estimated downtime: 30 seconds.
```

3. PostgreSQL Password Rotation (6 months)

3.1 Procedure

```
# 1. Generate new password
NEW_PG=$(openssl rand -base64 32)

# 2. Connect to DB and change password
docker compose exec db psql -U erp -d erp \
  -c "ALTER USER erp WITH PASSWORD '$NEW_PG';"

# 3. Update .env
sed -i "s|^POSTGRES_PASSWORD=.*|POSTGRES_PASSWORD=$NEW_PG|" backend/.env

# 4. Sync DATABASE_URL
sed -i "s|postgres://erp:[^@]*@|postgres://erp:$NEW_PG@" backend/.env

# 5. Restart backend (DB stays up)
docker compose restart backend
```

3.2 Verify

```
docker compose exec backend python -c "
import asyncio
from app.database import engine
async def t():
    async with engine.connect() as c:
        r = await c.exec_driver_sql('SELECT 1')
        print('DB OK:', r.scalar())
asyncio.run(t())
"
```

4. LLM_API_KEY Rotation

4.1 Trigger Conditions

1. Yearly schedule: every Jan 1
2. Emergency: abnormal usage / bill explosion / accidentally committed to GitHub

4.2 Emergency Rotation (15 minutes)

```
# 1. Immediately invalidate old key on provider console
#   - Anthropic: console.anthropic.com → API Keys → Delete
#   - OpenAI: platform.openai.com → API Keys → Revoke
#   - DeepSeek: platform.deepseek.com → API Keys

# 2. Generate new key

# 3. Update .env
sed -i "s|^LLM_API_KEY=.*|LLM_API_KEY=sk-new..." backend/.env

# 4. Restart
docker compose restart backend

# 5. Verify
curl -s http://localhost:8000/api/health | jq .llm_provider

# 6. Send 1 test chat
curl -X POST http://localhost:8000/api/chat-v2 \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"message":"test","session_id":"rotation-check"}'
```

4.3 Post-Check

```
# Check if old key still in use (logs should not show old key fingerprint)
docker compose logs backend --since 5m | grep -i "401\|auth.*fail"
```

5. TLS Certificate Rotation (90 days)

5.1 Let's Encrypt Fully Automated

```
# Install certbot if not yet
apt install certbot

# First-time
certbot certonly --webroot -w /var/www/certbot \
  -d your-domain.example -d www.your-domain.example \
  --email admin@your-domain.example --agree-tos

# Set up cron for auto-renew
echo '0 3 * * * certbot renew --quiet --post-hook "nginx -s reload"' \
  | sudo crontab -

# Test renew manually
certbot renew --dry-run
```

5.2 Manual Rotation (self-signed for internal use)

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/nginx/certs/erp.key \
  -out /etc/nginx/certs/erp.crt \
  -subj "/CN=erp.local"

# Reload nginx
docker compose exec frontend nginx -s reload
```

6. WireGuard Public Key Rotation (yearly)

6.1 Procedure

Annually rotate the keypair for HQ and all Factory Nodes synchronously.

```
# On each node (HQ + each factory)
wg genkey | tee privatekey | wg pubkey > publickey
# Record publickey content

# HQ side: update wg0.conf – change all factory peers to new publickeys
# Factory side: update wg0.conf – change HQ peer to new publickey

# Restart wireguard
wg-quick down wg0 && wg-quick up wg0
```

6.2 Verify

```
# On HQ
wg show
# All peers should show "latest handshake" within 1 minute
```

7. Emergency Response: Secret Leak

7.1 Detection

Signal	Action
GitGuardian alert	Immediately §7.2
LLM bill spike	Immediately §7.2
Secret seen in public channel	Immediately §7.2
Mass 401/403 from unknown IP	§7.3 whitelist
Anomalous session in audit log	§7.4 revoke

7.2 Golden 15 Minutes

Must do within 15 minutes of leak:

```
T+0min: Confirm leak scope (which secret? which commit?)
T+2min: Invalidate old key on provider console
T+5min: Generate new secret and write to .env
T+8min: docker compose restart backend
T+12min: Run smoke test to confirm new secret works
T+15min: Internal notification + (if applicable) customer notification
```

7.3 Add IP Whitelist

```
# nginx/conf.d/default.conf
location /api/ {
    # Emergency: only allow customer office IP
    allow 203.0.113.0/24; # customer office
    deny all;
    proxy_pass http://backend:8000;
}
```

7.4 Revoke Suspicious Sessions

```
# Change JWT_SECRET → all sessions invalidated
NEW=$(openssl rand -hex 32)
sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$NEW|" backend/.env
docker compose restart backend
```

7.5 Post-Mortem RCA

Document in `docs/incidents/YYYY-MM-DD-secret-leak.md`:

- Timeline
- Type of leaked secret
- Blast radius
- Remediation actions
- Preventive measures

8. Automation + Monitoring

8.1 Scheduled Reminders

Add to cron:

```
# Monthly check on 1st
0 9 1 * * /opt/llm-erp/scripts/secret_age_check.sh
```

`secret_age_check.sh`:


```
#!/bin/bash
# Calculate days since .env last modified (proxy for JWT_SECRET age)
LAST_CHANGE=$(stat -c %Y backend/.env)
NOW=$(date +%s)
AGE_DAYS=$(( (NOW - LAST_CHANGE) / 86400 ))

if [ $AGE_DAYS -gt 80 ]; then
    curl -X POST https://notify-api.line.me/api/notify \
        -H "Authorization: Bearer $LINE_TOKEN" \
        -d "message=⚠️ JWT_SECRET not rotated for $AGE_DAYS days, please see §2"
fi
```

8.2 GitGuardian Integration

Add to git pre-commit hook:

```
# .git/hooks/pre-commit
ggshield secret scan pre-commit
```

Ensure secrets are never committed.



Related Documents

- [Architecture Blueprint](#) (§5 Secrets Management overview)
- [Backup & Restore SOP](#) (includes .env backup)
- [Support Runbook](#)
- Chinese version: [SECRETS_ROTATION_SOP_ZH.md](#)

Version: 2.6 · Last updated: 2026-05-14